LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Az Zuhri Al Atsary (2509106072)

Kelas (B2 ‘25)

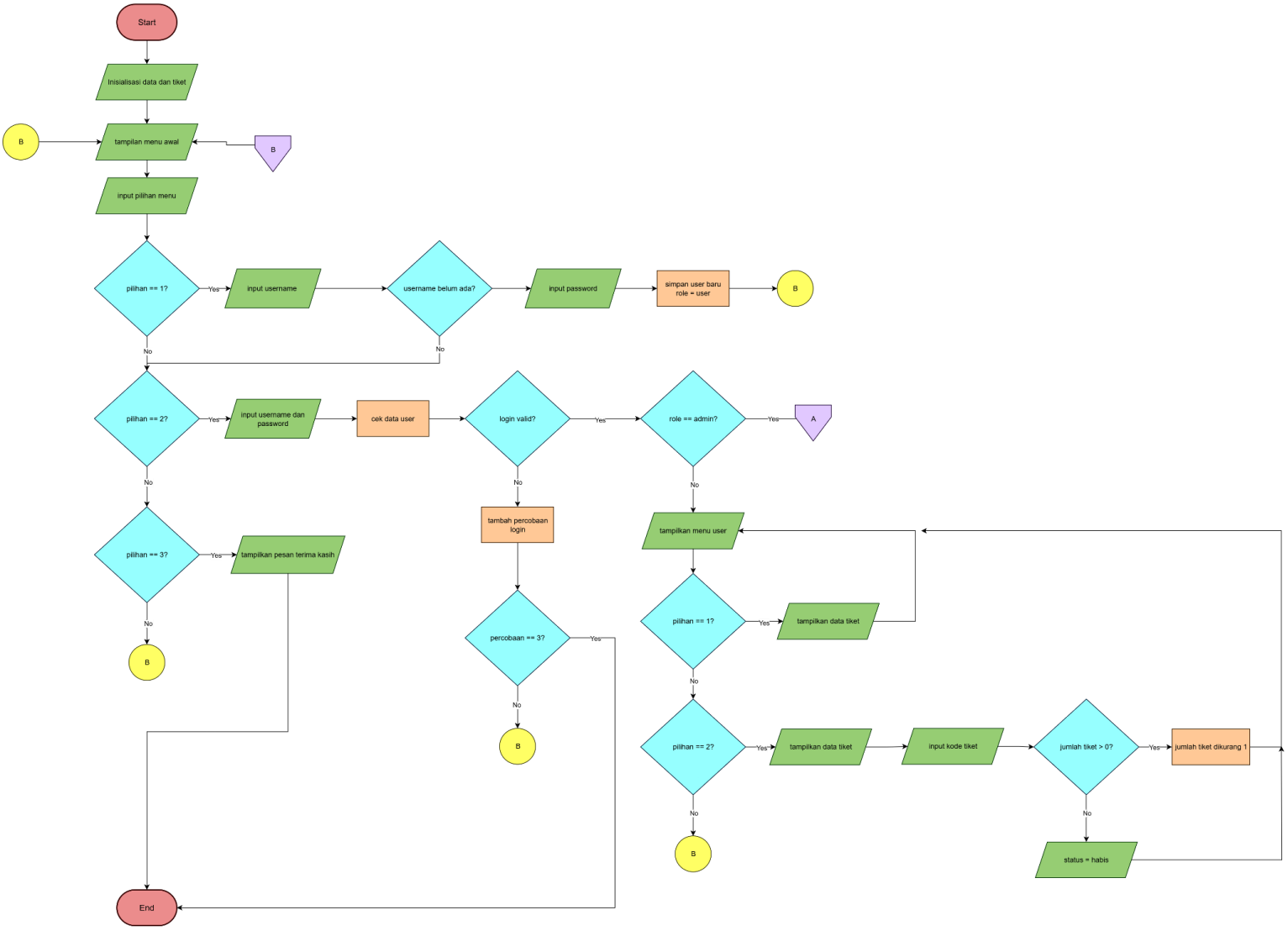
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

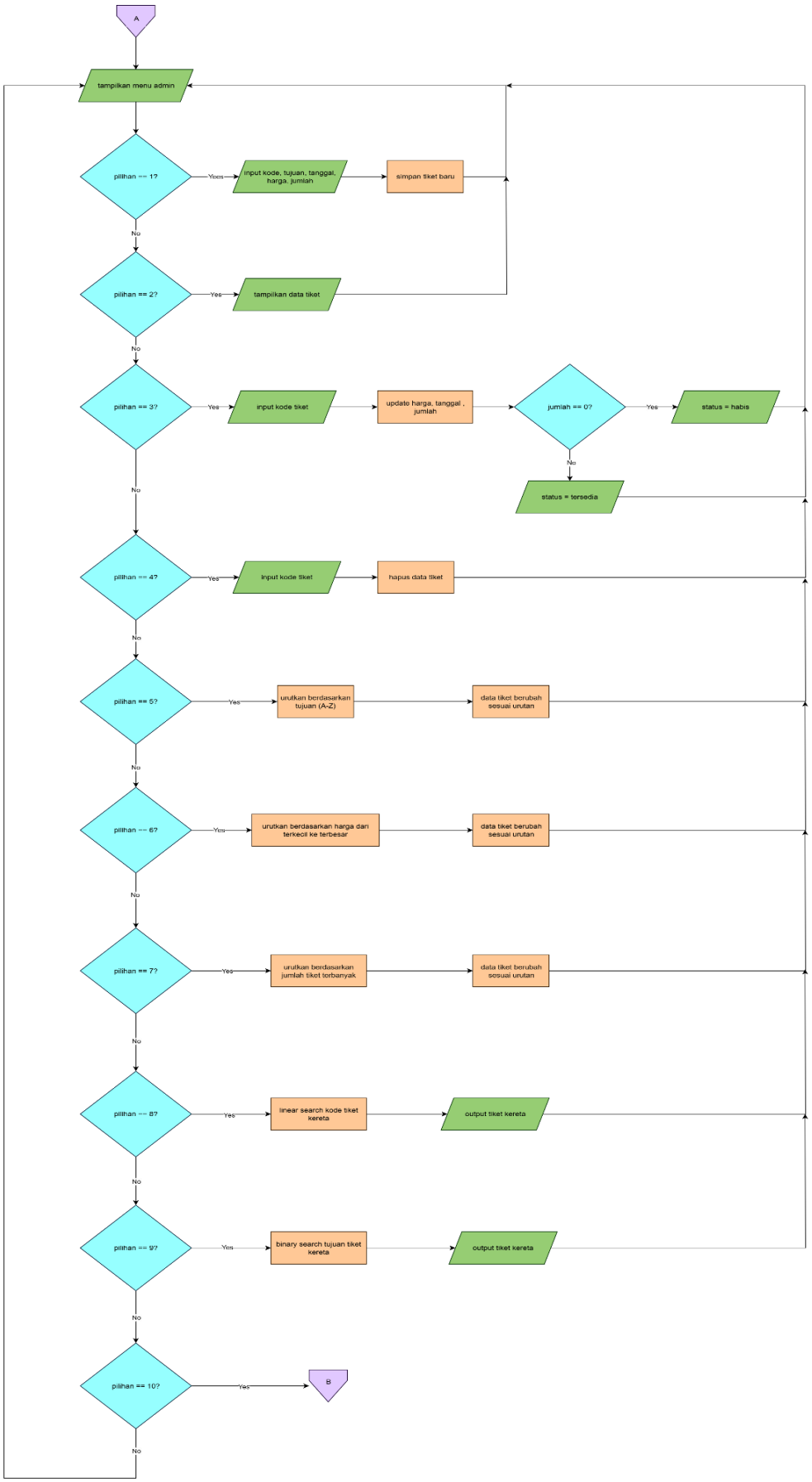
SAMARINDA

2026

1. Flowchart



Gambar 1.1 Flowchart pemesanan tiket kereta



Gambar 1.2 Flowchart pemesanan tiket kereta

Penjelasan Singkat Flowchart

1. START
2. Program menginisialisasi data user dan tiket yang sudah tersedia.
3. Program menampilkan menu awal:
    - Register
    - Login
    - Keluar
4. User memilih menu.
5. Jika memilih Register:
    - User memasukkan username dan password.
    - Data disimpan dengan role user.
    - Kembali ke menu awal.
6. Jika memilih Login:
    - User memasukkan username dan password.
    - Program memeriksa kecocokan data (maksimal 3 percobaan).
7. Jika login berhasil, program mengecek role user:
    - Admin → masuk ke menu admin.
    - User → masuk ke menu user.
8. Menu Admin:
    - Tambah tiket.
    - Lihat tiket.
    - Update tiket.
    - Hapus tiket.
    - Sort Tujuan (Z-A)
    - Sort Harga (Asc)
    - Sort Jumlah (Desc)
    - Search Kode (Linear)
    - Search Tujuan (Binary)
    - Logout kembali ke menu awal.
9. Menu User:
    - Lihat tiket.
    - Beli tiket (jumlah tiket berkurang, jika 0 maka status habis).
    - Logout kembali ke menu awal.
10. Program terus berjalan sampai user memilih Keluar pada menu awal.
11. END

2. Deskripsi Singkat Program

Program ini merupakan sistem sederhana untuk pengelolaan dan pembelian tiket dengan dua jenis pengguna, yaitu admin dan user. Pengguna dapat melakukan registrasi untuk membuat akun baru atau login dengan akun yang sudah ada, dengan batas maksimal tiga kali percobaan login.

Setelah berhasil login, pengguna akan diarahkan ke menu sesuai perannya. Admin dapat mengelola data tiket seperti menambah, melihat, memperbarui, menghapus, melakukan pengurutan, serta pencarian data tiket. Sementara itu, user hanya dapat melihat daftar tiket dan melakukan pembelian tiket.

Pada proses pembelian, sistem akan mengecek ketersediaan tiket. Jika masih tersedia, jumlah tiket akan berkurang, dan jika habis maka status tiket berubah menjadi habis. Program juga dilengkapi dengan penanganan kesalahan untuk memastikan input valid dan mencegah error saat program berjalan.

Program akan terus berjalan dengan menampilkan menu secara berulang hingga pengguna memilih logout atau keluar.

3. Source Code

A. Struktur Data Awal dan Library

```
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <stdexcept>
#include "validasi.h"
using namespace std;

#define MAX_USER 10
#define MAX_TIKET 100

struct Jadwal{
    string tujuan;
    string tanggal;
};

struct Tiket{
    string kode;
    Jadwal jadwal;
    int harga;
    int jumlah;
    string status;
};

struct User{
    string username;
    string password;
    string role;
};

```

Gambar 3.1 Struktur data awal dan library

B. Fungsi mengecek username

```
bool cekUsername(User user[], int jumlahUser, string username){
    for(int i=0;i<jumlahUser;i++){
        if(user[i].username == username){
            return true;
        }
    }
    return false;
}

```

Gambar 3.2 Fungsi untuk mengecek username

C. Fungsi login user

```
bool loginUser(User user[], int jumlahUser, string &roleLogin){
    string username,password;

    for(int i=1;i<=3;i++){
        system("cls");

        cout<<"============= LOGIN =============\n";
        cout<<"Username : ";
        cin>>username;
        cout<<"Password : ";
        cin>>password;

        for(int j=0;j<jumlahUser;j++){
            if(username == user[j].username && password == user[j].password){
                roleLogin = user[j].role;
                return true;
            }
        }

        cout<<"Login gagal\n";
        system("pause");
    }
    return false;
}
```

Gambar 3.3 Fungsi login user

D. Prosedur register

```
void registerUser(User user[], int &jumlahUser){
    system("cls");

    try{
        if(jumlahUser >= MAX_USER){
            throw length_error("Kapasitas user penuh!");
        }

        string username,password;
        cin.ignore();

        cout<<"=========== REGISTER USER ===========\n";
        cout<<"Username : ";
        getline(cin, username);
        cekKosong(username);

        if(cekUsername(user,jumlahUser,username)){
            throw runtime_error("Username sudah digunakan!");
        }

        cout<<"Password : ";
        getline(cin, password);
        cekKosong(password);

        user[jumlahUser].username = username;
        user[jumlahUser].password = password;
        user[jumlahUser].role = "user";

        jumlahUser++;

        cout<<"Register berhasil\n";
    }
    catch(const exception &e){
        cout<<"Error: "<<e.what()<<endl;
    }

    system("pause");
}
```

Gambar 3.4 prosedur register user

E. Prosedur menampilkan tiket

```
void tampilTiket(Tiket *tiket, int jumlahTiket){
    system("cls");
    cout<<"================ DATA TIKET ================\n";

    for(int i=0;i<jumlahTiket;i++){
        cout<<"Kode    : "<<tiket[i].kode<<endl;
        cout<<"Tujuan  : "<<tiket[i].jadwal.tujuan<<endl;
        cout<<"Tanggal : "<<tiket[i].jadwal.tanggal<<endl;
        cout<<"Harga   : "<<tiket[i].harga<<endl;
        cout<<"Jumlah  : "<<tiket[i].jumlah<<endl;
        cout<<"Status  : "<<tiket[i].status<<endl;
        cout<<"-------------------------------------------\n";
    }
}
```

Gambar 3.5 Prosedur untuk menampilkan tiket

F. Prosedur menambahkan tiket

```
void tambahTiket(Tiket *tiket, int *jumlahTiket){
    system("cls");

    try{
        if(*jumlahTiket >= MAX_TIKET){
            throw length_error("Kapasitas tiket penuh!");
        }

        cin.ignore();

        cout<<"Kode Tiket : ";
        getline(cin, tiket[*jumlahTiket].kode);
        cekKosong(tiket[*jumlahTiket].kode);

        cout<<"Tujuan : ";
        getline(cin, tiket[*jumlahTiket].jadwal.tujuan);

        cout<<"Tanggal : ";
        getline(cin, tiket[*jumlahTiket].jadwal.tanggal);

        cout<<"Harga : ";
        cin>>tiket[*jumlahTiket].harga;
        if(cin.fail()) throw invalid_argument("Harga harus angka!");
        cekAngka(tiket[*jumlahTiket].harga);

        cout<<"Jumlah tiket : ";
        cin>>tiket[*jumlahTiket].jumlah;
        cekAngka(tiket[*jumlahTiket].jumlah);

        tiket[*jumlahTiket].status = "tersedia";
        (*jumlahTiket)++;

        cout<<"Tiket berhasil ditambahkan\n";
    }
    catch(const exception &e){
        cout<<"Error: "<<e.what()<<endl;
        cin.clear();
        cin.ignore(1000,'\n');
    }

    system("pause");
}
```

Gambar 3.6 Prosedur untuk menambahkan tiket

G. Prosedur update tiket

```
void updateTiket(Tiket *tiket, int jumlahTiket){
    system("cls");

    try{
        string kode;
        cout<<"Masukkan kode tiket : ";
        cin>>kode;

        int i = linearSearchKode(tiket, jumlahTiket, kode);

        if(i == -1){
            throw out_of_range("Kode tiket tidak ditemukan!");
        }

        cout<<"Harga baru : ";
        cin>>tiket[i].harga;
        cekAngka(tiket[i].harga);

        cout<<"Tanggal baru : ";
        cin>>tiket[i].jadwal.tanggal;

        cout<<"Jumlah tiket : ";
        cin>>tiket[i].jumlah;
        cekAngka(tiket[i].jumlah);

        tiket[i].status = (tiket[i].jumlah == 0) ? "habis" : "tersedia";

        cout<<"Data berhasil diupdate\n";
    }
    catch(const exception &e){
        cout<<"Error: "<<e.what()<<endl;
    }

    system("pause");
}
```

Gambar 3.7 Prosedur untuk mengupdate tiket

H. Prosedur hapus tiket

```
void hapusTiket(Tiket *tiket, int *jumlahTiket){
    system("cls");

    try{
        string kode;
        cout<<"Masukkan kode tiket : ";
        cin>>kode;

        int i = linearSearchKode(tiket, *jumlahTiket, kode);

        if(i == -1){
            throw out_of_range("Kode tidak ditemukan!");
        }

        for(int j=i;j<*jumlahTiket-1;j++){
            tiket[j] = tiket[j+1];
        }
        (*jumlahTiket)--;

        cout<<"Data berhasil dihapus\n";
    }
    catch(const exception &e){
        cout<<"Error: "<<e.what()<<endl;
    }

    system("pause");
}
```

Gambar 3.8 Prosedur untuk menghapus tiket

I. Prosedur beli tiket

```
void beliTiket(Tiket *tiket, int jumlahTiket){
    system("cls");

    try{
        tampilTiket(tiket,jumlahTiket);

        string kode;
        cout<<"Masukkan kode tiket : ";
        cin>>kode;

        int i = linearSearchKode(tiket, jumlahTiket, kode);

        if(i == -1){
            throw out_of_range("Kode tidak ditemukan!");
        }

        if(tiket[i].jumlah <= 0){
            throw runtime_error("Tiket habis!");
        }

        tiket[i].jumlah--;
        tiket[i].status = (tiket[i].jumlah == 0) ? "habis" : "tersedia";

        cout<<"Tiket berhasil dibeli\n";
    }
    catch(const exception &e){
        cout<<"Error: "<<e.what()<<endl;
    }

    system("pause");
}
```

Gambar 3.9 Prosedur untuk membeli tiket

J. Prosedur menu admin

```
void menuAdmin(Tiket *tiket, int *jumlahTiket){
    int pilih;
    do{
        system("cls");

        cout<<"========== MENU ADMIN ==========\n";
        cout<<"1. Tambah Tiket\n";
        cout<<"2. Lihat Tiket\n";
        cout<<"3. Update Tiket\n";
        cout<<"4. Hapus Tiket\n";
        cout<<"5. Sort Tujuan (Z-A)\n";
        cout<<"6. Sort Harga (Asc)\n";
        cout<<"7. Sort Jumlah (Desc)\n";
        cout<<"8. Search Kode\n";
        cout<<"9. Search Tujuan\n";
        cout<<"10. Logout\n";
        cout<<"Pilih : ";
        cin>>pilih;

        if(cin.fail()){
            cin.clear();
            cin.ignore(1000,'\n');
            cout<<"Input harus angka!\n";
            system("pause");
            continue;
        }

        if(pilih==1) tambahTiket(tiket, jumlahTiket);
        else if(pilih==2){ tampilTiket(tiket,*jumlahTiket); system("pause"); }
        else if(pilih==3) updateTiket(tiket,*jumlahTiket);
        else if(pilih==4) hapusTiket(tiket,jumlahTiket);
        else if(pilih==5) sortTujuanDesc(tiket,*jumlahTiket);
        else if(pilih==6) sortHargaAsc(tiket,*jumlahTiket);
        else if(pilih==7) sortJumlahDesc(tiket,*jumlahTiket);

        else if(pilih==8){
            try{
                string kode;
                cout<<"Masukkan kode tiket : ";
                cin>>kode;

                int i = linearSearchKode(tiket,*jumlahTiket,kode);

                if(i == -1) throw out_of_range("Data tidak ditemukan!");

                cout<<"Data ditemukan:\n";
                cout<<"Kode    : "<<tiket[i].kode<<endl;
                cout<<"Tujuan  : "<<tiket[i].jadwal.tujuan<<endl;
                cout<<"Tanggal : "<<tiket[i].jadwal.tanggal<<endl;
                cout<<"Harga   : "<<tiket[i].harga<<endl;
                cout<<"Jumlah  : "<<tiket[i].jumlah<<endl;
                cout<<"Status  : "<<tiket[i].status<<endl;
            }
            catch(const exception &e){
                cout<<"Error: "<<e.what()<<endl;
            }
            system("pause");
        }

        else if(pilih==9){
            try{
                string tujuan;
                cout<<"Masukkan tujuan : ";
                cin>>tujuan;

                sortTujuanAsc(tiket,*jumlahTiket);
                int i = binarySearchTujuan(tiket,*jumlahTiket,tujuan);

                if(i == -1) throw out_of_range("Data tidak ditemukan!");

                cout<<"Data ditemukan:\n";
                cout<<"Kode    : "<<tiket[i].kode<<endl;
                cout<<"Tujuan  : "<<tiket[i].jadwal.tujuan<<endl;
                cout<<"Tanggal : "<<tiket[i].jadwal.tanggal<<endl;
                cout<<"Harga   : "<<tiket[i].harga<<endl;
                cout<<"Jumlah  : "<<tiket[i].jumlah<<endl;
                cout<<"Status  : "<<tiket[i].status<<endl;
            }
            catch(const exception &e){
                cout<<"Error: "<<e.what()<<endl;
            }
            system("pause");
        }

    }while(pilih!=10);
}
```

Gambar 3.10 Prosedur menu admin

K. Prosedur menu user

```
void menuUser(Tiket *tiket, int jumlahTiket){
    system("cls");

    int pilih;
    do{
        system("cls");

        cout<<"========== MENU USER ==========\n";
        cout<<"1. Lihat Tiket\n";
        cout<<"2. Beli Tiket\n";
        cout<<"3. Logout\n";
        cout<<"Pilih : ";
        cin>>pilih;

        if(pilih==1){
            tampilTiket(tiket,jumlahTiket);
            system("pause");
        }
        else if(pilih==2){
            beliTiket(tiket,jumlahTiket);
        }

    }while(pilih!=3);
}
```

Gambar 3.11 Prosedur menu user

L. Fungsi utama

```
int main(){
    User user[MAX_USER];
    Tiket tiket[MAX_TIKET];

    int jumlahUser = 2;
    int jumlahTiket = 2;

    user[0]={"admin","admin","admin"};
    user[1]={"user","user","user"};

    tiket[0]={"TK01",{"Jakarta","10-06-2026"},200000,10,"tersedia"};
    tiket[1]={"TK02",{"Bandung","12-06-2026"},150000,5,"tersedia"};
    tiket[2]={"TK03",{"Surabaya","15-06-2026"},180000,8,"tersedia"};

    int menu;
    do{
        system("cls");

        cout<<"========== MENU UTAMA ==========\n";
        cout<<"1. Register\n";
        cout<<"2. Login\n";
        cout<<"3. Keluar\n";
        cout<<"Pilih : ";
        cin>>menu;

        if(menu==1){
            registerUser(user,jumlahUser);
        }
        else if(menu==2){
            string role;

            if(loginUser(user,jumlahUser,role)){
                if(role=="admin"){
                    menuAdmin(tiket,&jumlahTiket);
                }else{
                    menuUser(tiket,jumlahTiket);
                }
            }else{
                system("pause");
            }
        }

    }while(menu!=3);
}
```

Gambar 3.12 Fungsi utama atau main program

M. Sorting tujuan

```
void sortTujuanDesc(Tiket *tiket, int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(tiket[j].jadwal.tujuan < tiket[j+1].jadwal.tujuan){
                swap(tiket[j], tiket[j+1]);
            }
        }
    }
    cout<<"Sorting tujuan (Z-A) selesai\n";
    system("pause");
}
```

Gambar 3.13 Sort tujuan descending

N. Sorting harga

```
void sortHargaAsc(Tiket *tiket, int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(tiket[j].harga > tiket[j+1].harga){
                swap(tiket[j], tiket[j+1]);
            }
        }
    }
    cout<<"Sorting harga (kecil-besar) selesai\n";
    system("pause");
}
```

Gambar 3.14 Sorting harga ascending

L. Sorting jumlah tiket

```
void sortJumlahDesc(Tiket *tiket, int n){
    for(int i=0;i<n-1;i++){
        for(int j=0;j<n-i-1;j++){
            if(tiket[j].jumlah < tiket[j+1].jumlah){
                swap(tiket[j], tiket[j+1]);
            }
        }
    }
    cout<<"Sorting jumlah (terbanyak) selesai\n";
    system("pause");
}
```

Gambar 3.15 sorting jumlah tiket

M. Linear Search

```
int linearSearchKode(Tiket *tiket, int n, string kode){
    for(int i=0;i<n;i++){
        if(tiket[i].kode == kode){
            return i;
        }
    }
    return -1;
}
```

Gambar 3.16 linear search kode

N. Binary Search

```
int binarySearchTujuan(Tiket *tiket, int n, string tujuan){
    int left = 0, right = n - 1;

    while(left <= right){
        int mid = (left + right) / 2;

        if(tiket[mid].jadwal.tujuan == tujuan){
            return mid;
        }
        else if(tiket[mid].jadwal.tujuan < tujuan){
            left = mid + 1;
        }
        else{
            right = mid - 1;
        }
    }
    return -1;
}
```

Gambar 3.17 Fungsi utama atau main program

4. Hasil Output

```
========== MENU UTAMA ==========
1. Register
2. Login
3. Keluar
Pilih :
```

Gambar 4.1 Tampilan menu awal

```
=========== REGISTER USER ===========
Username : hebattt
Password : 1234
Register berhasil
Press any key to continue . . .
```

Gambar 4.2 Tampilan register

```
============= LOGIN ==============
Username : hebattt
Password : 1234
```

Gambar 4.3 Tampilan login

```
========== MENU USER ==========
1. Lihat Tiket
2. Beli Tiket
3. Logout
Pilih :
```

Gambar 4.4 Tampilan menu user

```
========== MENU ADMIN ==========
1. Tambah Tiket
2. Lihat Tiket
3. Update Tiket
4. Hapus Tiket
5. Sort Tujuan (Z-A)
6. Sort Harga (Asc)
7. Sort Jumlah (Desc)
8. Search Kode (Linear)
9. Search Tujuan (Binary)
10. Logout
Pilih :
```

Gambar 4.5 Tampilan menu admin

```
================ DATA TIKET ================
Kode    : TK01
Tujuan  : Jakarta
Tanggal : 10-06-2026
Harga   : 200000
Jumlah  : 10
Status  : tersedia
--------------------------------------------
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Press any key to continue . . .
```

Gambar 4.6 Tampilan data tiket

```
================ DATA TIKET ================
Kode    : TK01
Tujuan  : Jakarta
Tanggal : 10-06-2026
Harga   : 200000
Jumlah  : 10
Status  : tersedia
--------------------------------------------
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Kode    : TK03
Tujuan  : Malang
Tanggal : 26-06-2026
Harga   : 250000
Jumlah  : 10
Status  : tersedia
--------------------------------------------
Press any key to continue . . .
```

Gambar 4.7 Tampilan data tiket ketika ditambahkan

```
================ DATA TIKET ================
Kode    : TK01
Tujuan  : Jakarta
Tanggal : 10-06-2026
Harga   : 200000
Jumlah  : 10
Status  : tersedia
--------------------------------------------
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Kode    : TK03
Tujuan  : Malang
Tanggal : 04-04-2026
Harga   : 100000
Jumlah  : 13
Status  : tersedia
--------------------------------------------
Press any key to continue . . .
```

Gambar 4.8 Tampilan data tiket ketika diperbarui

```
================ DATA TIKET ================
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Kode    : TK03
Tujuan  : Malang
Tanggal : 04-04-2026
Harga   : 100000
Jumlah  : 13
Status  : tersedia
--------------------------------------------
Press any key to continue . . .
```

Gambar 4.9 Tampilan data tiket ketika dihapus

```
================ DATA TIKET ================
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Kode    : TK03
Tujuan  : Malang
Tanggal : 04-04-2026
Harga   : 100000
Jumlah  : 13
Status  : tersedia
--------------------------------------------
Masukkan kode tiket : TK02
Tiket berhasil dibeli
Press any key to continue . . .
```

Gambar 4.10 Tampilan ketika tiket berhasil dibeli

```
============= LOGIN ==============
Username : zuhri
Password : 111111111111111111111111

Login gagal
Press any key to continue . . .
```

Gambar 4.11 Tampilan ketika gagal login

```
================ DATA TIKET ================
Kode    : TK03
Tujuan  : Samarinda
Tanggal : 26-07-2026
Harga   : 350000
Jumlah  : 15
Status  : tersedia
--------------------------------------------
Kode    : TK01
Tujuan  : Jakarta
Tanggal : 10-06-2026
Harga   : 200000
Jumlah  : 10
Status  : tersedia
--------------------------------------------
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Press any key to continue . . .
```

Gambar 4.12 Tampilan urutan berdasarkan tujuan

```
================ DATA TIKET ================
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Kode    : TK01
Tujuan  : Jakarta
Tanggal : 10-06-2026
Harga   : 200000
Jumlah  : 10
Status  : tersedia
--------------------------------------------
Kode    : TK03
Tujuan  : Samarinda
Tanggal : 26-07-2026
Harga   : 350000
Jumlah  : 15
Status  : tersedia
--------------------------------------------
Press any key to continue . . .
```

Gambar 4.13 Tampilan urutan berdasarkan harga

```
================ DATA TIKET ================
Kode    : TK03
Tujuan  : Samarinda
Tanggal : 26-07-2026
Harga   : 350000
Jumlah  : 15
Status  : tersedia
--------------------------------------------
Kode    : TK01
Tujuan  : Jakarta
Tanggal : 10-06-2026
Harga   : 200000
Jumlah  : 10
Status  : tersedia
--------------------------------------------
Kode    : TK02
Tujuan  : Bandung
Tanggal : 12-06-2026
Harga   : 150000
Jumlah  : 5
Status  : tersedia
--------------------------------------------
Press any key to continue . . .
```

Gambar 4.14 Tampilan ketika tiket berhasil dibeli

```
Masukkan kode tiket : TK01
Data ditemukan:
Kode    : TK01
Tujuan  : Jakarta
Tanggal : 10-06-2026
Harga   : 200000
Jumlah  : 10
Status  : tersedia
Press any key to continue . . .
```

Gambar 4.15 Tampilan menggunakan linear search

```
Pilih : 9
Masukkan tujuan : Surabaya
Data ditemukan:
Kode     : TK03
Tujuan   : Surabaya
Tanggal  : 15-06-2026
Harga    : 180000
Jumlah   : 8
Status   : tersedia
Press any key to continue . . .
```

Gambar 4.16 Tampilan menggunakan binary search

```
=========== REGISTER USER ===========
Username : zuhri
Error: Username sudah digunakan!
Press any key to continue . . .
```

Gambar 4.17 Tampilan ketika username sudah digunakan

```
=========== REGISTER USER ===========
Username :
Error: Input tidak boleh kosong!
Press any key to continue . . .
```

Gambar 4.18 Tampilan ketika input username kosong

5. Langkah-langkah GIT

5.1 GIT Init

```
PS C:\Users\azzuh\OneDrive\Documents\Praktikum-APL> git init
Reinitialized existing Git repository in C:/Users/azzuh/OneDrive/Documents/Praktikum-APL/.git/
```

Gambar 5.1
Membuat repository Git baru di folder proyek

5.2 GIT Add

```
PS C:\Users\azzuh\OneDrive\Documents\Praktikum-APL> git add .
```

Gambar 5.2
Menandai semua perubahan file di folder agar siap disimpan (commit) ke Git.

5.3 GIT Commit

```
PS C:\Users\azzuh\OneDrive\Documents\Praktikum-APL> git commit -m "Finish Post Test 7"
[main 4e067f7] Finish Post Test 7
 5 files changed, 541 insertions(+)
 create mode 100644 Kelas/Pertemuan-7/Pertemuan-7.cpp
 create mode 100644 Kelas/Pertemuan-7/Pertemuan-7.exe
 create mode 100644 Post-Test/Post-Test-APL-7/2509106072-AZZUHRIALATSARY-PT-7/main.cpp
 create mode 100644 Post-Test/Post-Test-APL-7/2509106072-AZZUHRIALATSARY-PT-7/main.exe
 create mode 100644 Post-Test/Post-Test-APL-7/2509106072-AZZUHRIALATSARY-PT-7/validasi.h
PS C:\Users\azzuh\OneDrive\Documents\Praktikum-APL>
```

Gambar 5.3
Menyimpan perubahan yang sudah ditandai ke dalam riwayat Git, dengan pesan sebagai catatan perubahan.

5.4 GIT Push

```
PS C:\Users\azzuh\OneDrive\Documents\Praktikum-APL> git push -u origin main
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 20 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 958.36 KiB | 6.94 MiB/s, done.
Total 12 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 2 local objects.
To https://github.com/azzuhrialatsary-dev/praktikum-apl.git
   cbd06e4..4e067f7  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\azzuh\OneDrive\Documents\Praktikum-APL>
```

Gambar 5.4
Mengirim commit (perubahan yang sudah disimpan) dari repositori lokal ke repositori remote seperti GitHub.